Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Колоризация изображений с помощью нейронных сетей

Данная лабораторная работа посвящена автоматической колоризации изображений (преобразование изображения из черно-белого вида в цветной) с помощью нейронных сетей.

Цель работы: получить навыки автоматического преобразования черно-белых изображений в цветные с помощью нейронных сетей.

Задачи:

1. Ознакомиться с работой сверточных нейронных сетей в библиотеке Keras;
2. Научиться решать задачу колоризации изображений с помощью сверточных нейронных сетей.
3. Научиться применять модель сверточной нейронной сети U-Net для колоризации изображений.

Колоризация изображений

Колоризация – преобразование монохромного (чёрно-белого) кинематографического или фотоизображения в цветное (рис. 1). Колоризация считается разновидностью цветного кинематографа или фотографии и применялась до изобретения этих технологий. В настоящее время колоризация находит применение при реставрации кинофильмов для их улучшения.



Рисунок 1. Пример колоризации изображения

До появления цифровых технологий колоризация черно-белых фотографий осуществлялась вручную. Бумажные фотографии раскрашивались акварелью, масляными красками, цветными карандашами и пастелью. Инструментами служили кисть или распылитель. Популярность раскрашивания пришлась на середину XIX – начало XX века, пока на рынке не появились многослойные цветные фотоматериалы, технология окрашивания фотохром. Сегодня колоризация фотографий в цифровом виде осуществляется с помощью методов искусственного интеллекта и нейронных сетей.

Ход работы

1. Загрузка данных

В качестве изображений будет использоваться набор данных Flickr30k. Изображения представляют сбой различные сцены с людьми, животными, видами природы, архитектуры и т.д. Данный набор изображений размещен на сайте Kaggle и его можно скачать по ссылке: https://www.kaggle.com/datasets/hsankesara/flickr-image-dataset



Рисунок 2. Пример изображений Flickr30k

Данный набор содержит около 30000 изображений и объем 9 Гб. Для нашей работы данное количество изображений является избыточным в виду отсутствия необходимых компьютерных мощностей. Поэтому для обучения нейронной сети мы выбрали из этого датасета всего 5000 изображений и загрузили их на Google диск. Вы можете скачать набор изображений по ссылке: https://drive.google.com/drive/folders/1jVqX_gm0569IHKdsZJAxE_FRUjPKU3Qn?usp=sharing

Для экономии времени лучше скачивать архив формата zip. Далее Вам потребуется загрузить архив на свой Google диск и там его распаковать с помощью встроенного архиватора. Для обучения сети нам необходимо будет преобразовать изображения в черно-белый вид. Обучение будем выполнять путем подачи на нейронную сеть черно-белого изображения, а на выходе будем предъявлять его исходный цветной вариант.

Для начала работы выполним загрузку необходимых библиотек.

```
from sklearn.model_selection import train_test_split
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt
```

```
from skimage import io
import numpy as np
import random
import cv2
import os

from google.colab import drive
drive.mount('/content/drive')
```

Для загрузки датасета из Google диска выполните следующий код:

```
def download_data(path):
  data = []
  for path_image in sorted(os.listdir(path=path)):
      image = Image.open(path + path_image)
      data.append(np.array(image))
  return data

image_files = download_data(r"./drive/My Drive/Колоризация/Колоризация/")
```

2. Предобработка изображений

Для обучения нейронной сети нам потребуются изображения в виде черно-белого и цветного варианта. Создадим функцию **load_images_from_filelist**, которая загружает изображения из указанной папки, обрабатывает их и возвращает два массива: один с изображениями в градациях серого, а другой – с цветными изображениями.

```
def load_images_from_filelist(file_list, img_size=(128, 128),
.
.
Допишите код
.
.
```

file_list – путь к папке, из которой будут загружаться изображения.

img_size – размер, до которого будут изменены изображения (по умолчанию (128x128)).

max_samples – максимальное количество изображений, которые будут загружены (по умолчанию 10000).

images_gray – список для хранения изображений в градациях серого.

images_color – список для хранения цветных изображений.

img_np = np.array(img_pil) – конвертирует PIL-изображение в массив NumPy.

img = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR) – меняет цветовой канал с RGB (используется в PIL) на BGR (используется в OpenCV).

img_resized = cv2.resize(img, img_size) – приводит изображение к заданному размеру (по умолчанию 128×128).

img_gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY) – конвертирует цветное изображение в оттенки серого.

img_gray = img_gray.astype('float32') / 255.0 – нормализует значения пикселей в диапазон [0, 1] (из uint8 [0, 255] в float32 [0.0, 1.0]).

images_gray.append(img_gray) – сохраняет черно-белое изображение.

images_color.append(img_color) – сохраняет цветное изображение.

Для обучения нейросети распределим изображения по двум спискам x_data и y_data. Далее разделим эти списки на обучающую и тестовую выборки: 80% – обучающая выборка, 20% – тестовая выборка. Так же добавим дополнительное измерение к изображениям, чтобы соответствовать ожидаемому формату входных данных для нейронной сети.

```
.
.
Допишите код
.
.
```

Выведем на экран загруженные изображения.

```
random_indices = random.sample(range(len(x_test)), 4)

fig, axes = plt.subplots(2, 4, figsize=(9, 5))

for i, idx in enumerate(random_indices):
    original_image = cv2.cvtColor(y_test[idx], cv2.COLOR_BGR2RGB)
    axes[1, i].imshow(original_image)
    axes[1, i].set_title(f"Оригинал {idx}")
    axes[1, i].axis('off')

for i, idx in enumerate(random_indices):
    gray_image = x_test[idx].reshape(128, 128)
    axes[0, i].imshow(gray_image, cmap='gray')
    axes[0, i].set_title(f"Черно-белое {idx}")
    axes[0, i].axis('off')

plt.tight_layout()
plt.show()
```

На рисунке 3 вы можете видеть примеры загруженных изображений.



Рисунок 3. Примеры загруженных изображений

3. Реализация топологии нейронной сети

В данной работе в качестве архитектуры нейронной сети можно использовать U-Net подобную сеть, или обычный автоэнкодер. Оба типа этих сетей содержат сжимающий и расширяющий пути. В нашем случае на вход сети будет поступать черно-белое изображение, а на выходе сети будет предъявляться его цветной вариант.

На каждом слое увеличивается количество карт признаков. Сжимающий путь похож на типичную свёрточную сеть, он состоит из трех блоков. Каждый блок содержит два слоя свертки (Conv2D), после каждого из них идут слои BatchNormalization() и в конце каждого блока слой MaxPooling2D.

После сжимающего пути следует расширяющий путь, который состоит также из трех блоков. Каждый блок содержит слой обратной свертки Conv2DTranspose, потом идет слой BatchNormalization() и третьим идет слой UpSampling2D, который выполняет обратную операцию MaxPooling2D.

Последний слой Conv2DTranspose содержит 3 карты признаков, по одной карте на каждый канал цветного изображения.

```
.
.
Допишите код
.
.
```

Выберем параметры обучения и скомпилируем модель.

```
.
.
Допишите код
.
.
```

Conv2D – создание сверточного слоя со следующими параметрами: количество плоскостей (фильтров), размер ядра свертки, **activation** – функция активации, **padding** – параметр "same" означает не изменять размер изображения после его обработки сверточным слоем.

MaxPooling2D – создание слоя Max-pooling. Окном 2x2 сканируется предыдущий слой, выбирает в каждой такой области максимальный элемент и записывает его в текущий слой. Уменьшает размер входного слоя в 2 раза.

UpSampling2D – операция обратная Max-pooling. Слой увеличивается в 2 раза относительно предыдущего слоя.

Conv2DTranspose – создание слоя с обратной функцией свертки. Другими словами, это слой развертки.

ReLU – функция активации, выполняющая простое нелинейное преобразование поданных на вход данных (x). Возвращает x, если x > 0 и 0 в противном случае. Отличается высокой скоростью вычисления.

Sigmoid – функция активации, выполняющая нелинейное преобразование на основе математической функции.

4. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод .**fit()**, используя блок кода ниже:

```
.
.
Допишите код
.
.
```

5. Тестирование нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

6. Результат работы нейронной сети

Выберем несколько случайных изображений из тестового набора и выведем на экран результат их колоризации.

```
random_indices = random.sample(range(len(x_test)), 4)

fig, axes = plt.subplots(3, 4, figsize=(11, 7))

for i, idx in enumerate(random_indices):
    gray_image = x_test[idx].reshape(128, 128)
    axes[0, i].imshow(gray_image, cmap='gray')
    axes[0, i].set_title(f"Черно-белое {idx}")
    axes[0, i].axis('off')
```

```
for i, idx in enumerate(random_indices):
    original_image = cv2.cvtColor(y_test[idx], cv2.COLOR_BGR2RGB)
    axes[1, i].imshow(original_image)
    axes[1, i].set_title(f"Оригинал {idx}")
    axes[1, i].axis('off')

out = model.predict(np.array([x_test[idx] for idx in random_indices]))
for i, idx in enumerate(random_indices):
    predicted_image = cv2.cvtColor(out[i], cv2.COLOR_BGR2RGB)
    axes[2, i].imshow(predicted_image)
    axes[2, i].set_title(f"Предсказанное {idx}")
    axes[2, i].axis('off')

plt.tight_layout()
plt.show()
```

На рисунке 4 представлены результаты колоризации изображений. Как видно по полученным результатам нейронная сеть частично окрасила изображения, некоторые предметы на изображениях окрашены в другие цвета. Результат можно назвать удовлетворительным. Для улучшения результата необходимо увеличить количество эпох обучения, расширить обучающую выборку, а также оптимизировать архитектуру нейронной сети.

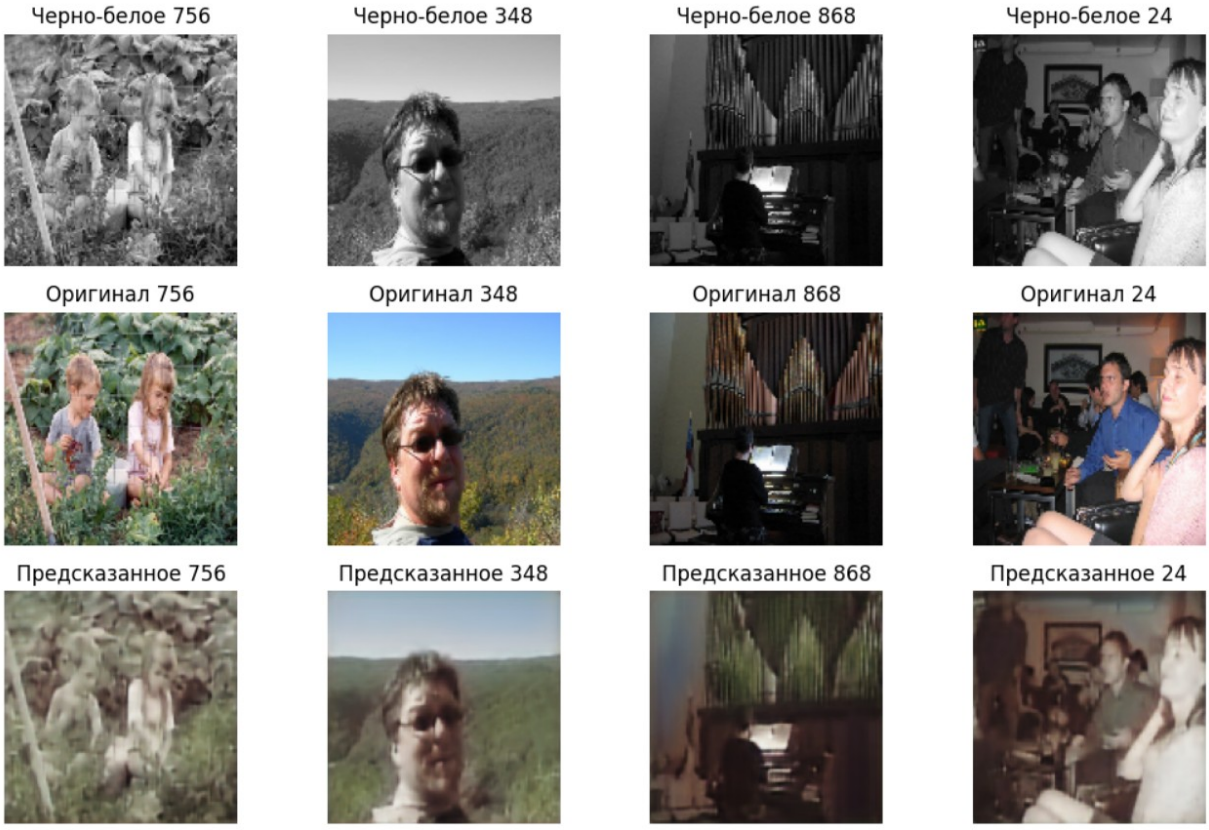


Рисунок 4. Пример колоризации изображений

Вывод

В результате выполнения данной лабораторной работы Вы получили навыки автоматического преобразования черно-белых изображений в цветные с помощью нейронных сетей и библиотеки машинного обучения Keras.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания - 0 баллов.

Большей частью выполнено - 2 балл.

Выполнено полностью - 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету - 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований - 2 балл.

Соответствует всем требованиям - 4 балла.